Below is a **complete and interview-level explanation** of **== vs equals() vs hashCode()**, with **examples**, **when to use what**, and the **exact contract rules** — plus a **master table** covering **Object, String, StringBuilder, StringBuffer, and major Collection classes**.

---

1) == vs equals() vs hashCode() (Concept)

Feature	**==**	**equals()**	**hashCode()**
Type	Operator	Method	Method
Purpose	Compares **references (memory address)**	Compares **logical equality (content)**	Gives **integer hash value** for object
Works on	Primitive + Object	Object	Object
For primitives	Compares value	❌ not applicable	❌ not applicable
For objects	Compares *same object?*	Compares *equal meaning?*	Used in hashing (HashMap/HashSet)
Used mostly in	Reference comparison	Business equality	Collections hashing

---

2) Real Meaning

✅ == for Objects

```
String a = new String("A");
String b = new String("A");

System.out.println(a == b);       // false (different objects)
```

✅ equals() for Objects

```
System.out.println(a.equals(b));   // true (same content)
```

✅ hashCode()

```
System.out.println(a.hashCode());  // same number for equal strings
System.out.println(b.hashCode());  // same
```

---

3) When to Use What (Rule of Thumb)

Situation	Use
You want to check same object reference	==
You want to check same content / data	equals()
You store objects in HashMap / HashSet / Hashtable	must implement both equals() + hashCode()

---

4) Contract Between equals() and hashCode() ✅

Official Contract:

1. If two objects are equal using equals(), they **must have same hashCode()**
2. if(a.equals(b)) => a.hashCode() == b.hashCode()
3. If hashCode is same, equals **may or may not be true**
   (hash collision possible)

---

5) BEST Interview Example (Breaking HashMap)

❌ Wrong Class (equals only but no hashCode)

```
class Employee {
   int id;
   Employee(int id){ this.id = id; }

   @Override
   public boolean equals(Object o) {
      if(this == o) return true;
      if(!(o instanceof Employee)) return false;
      return this.id == ((Employee)o).id;
   }
}
```

Issue:

```
HashSet<Employee> set = new HashSet<>();
set.add(new Employee(1));

System.out.println(set.contains(new Employee(1))); // false □
```

✅ Why? Because hashCode differs → stored in different bucket.

✅ Correct Class (equals + hashCode)

```
class Employee {
   int id;
   Employee(int id){ this.id = id; }

   @Override
   public boolean equals(Object o) {
      if(this == o) return true;
      if(!(o instanceof Employee)) return false;
      return this.id == ((Employee)o).id;
   }

   @Override
   public int hashCode() {
      return Integer.hashCode(id);
   }
}
```

Now:

```
System.out.println(set.contains(new Employee(1))); // true ✅
```

---

6) Object Class Default Behavior

Method	Default behavior in Object
equals()	same as == (reference equality)
hashCode()	based on memory identity (internal address-like)

---

7) Equals + HashCode Contract for: Object, String, StringBuilder, StringBuffer

✅ Master Table

Class	== checks	equals() behavior	hashCode() behavior	Contract maintained?
Object	reference	reference (same as ==)	identity-based	✅ yes
String	reference	✅ content-based (value equality)	✅ content-based	✅ yes
StringBuilder	reference	❌ not overridden → reference equality	identity hash (Object)	✅ (but not value-based)
StringBuffer	reference	❌ not overridden → reference equality	identity hash (Object)	✅ (but not value-based)

Important Note:

```
StringBuilder sb1 = new StringBuilder("Hi");
StringBuilder sb2 = new StringBuilder("Hi");

System.out.println(sb1.equals(sb2));   // false ✕
System.out.println(sb1.toString().equals(sb2.toString())); // true ✓
```

✓ So for StringBuilder/StringBuffer, use:

```
sb1.toString().equals(sb2.toString())
```

---

8) Contract in Collections (All Major Classes Table)

A) Collections that use hashCode() and equals() heavily

These are called **Hash-based collections**.

Collection Class	Uses hashCode?	Uses equals?	Why?
HashMap	✓ yes	✓ yes	key hashing and equality
HashSet	✓ yes	✓ yes	internally uses HashMap
LinkedHashMap	✓ yes	✓ yes	preserves insertion order + hashing
LinkedHashSet	✓ yes	✓ yes	preserves insertion order
Hashtable	✓ yes	✓ yes	synchronized HashMap
ConcurrentHashMap	✓ yes	✓ yes	thread-safe hashing

✓ For these:
If you override equals() you must override hashCode()

B) Collections that use equals() but not hashCode()

Collection Class	Uses equals?	Uses hashCode?	How it checks?
ArrayList	✓ yes	✕ no	linear search
LinkedList	✓ yes	✕ no	linear search

Vector	✅ yes	❌ no	linear search
Stack	✅ yes	❌ no	linear search

Example:

```
List<String> list = new ArrayList<>();
list.add("A");
System.out.println(list.contains("A")); // uses equals()
```

C) Sorted collections (uses compareTo / Comparator, not equals mainly)

Collection	Uses compareTo/Comparator?	Uses equals?	Uses hashCode?
TreeMap	✅ yes	indirectly	❌ no
TreeSet	✅ yes	indirectly	❌ no

Most Important Rule:

For TreeSet/TreeMap:
✅ **Uniqueness is decided by compareTo() == 0**, not equals().

```
TreeSet<String> ts = new TreeSet<>();
ts.add("A");
ts.add("A");
System.out.println(ts.size()); // 1
```

D) Queue / Deque

Class	Uses equals?	Uses hashCode?
PriorityQueue	not for ordering	❌
ArrayDeque	yes for contains/remove	❌

---

9) Full "Individual Collections" Equality + Hashing Table ✅

Type	Classes	Equality mechanism
List	ArrayList, LinkedList, Vector, Stack	uses equals()

Set (Hash)	HashSet, LinkedHashSet	uses hashCode() + equals()
Set (Sorted)	TreeSet	uses compareTo() / Comparator
Map (Hash)	HashMap, LinkedHashMap, Hashtable, ConcurrentHashMap	keys use hashCode() + equals()
Map (Sorted)	TreeMap	keys use compareTo() / Comparator
Queue	PriorityQueue	ordering via Comparator; equality rarely used

---

10) Best Practical Real-Time Usage Guide ✅

Real world case	**Use**
Comparing two integers / primitives	==
Comparing two object references	==
Comparing two String contents	equals()
Comparing String ignoring case	equalsIgnoreCase()
Comparing custom class objects by data fields	override equals() + hashCode()
Using in HashMap key / HashSet element	MUST override both
Using in TreeMap/TreeSet keys	implement Comparable / Comparator
Comparing StringBuilder/StringBuffer values	convert to String

---

11) One-Line Interview Conclusion ✅

✅ == → same object?
✅ equals() → same data/value?
✅ hashCode() → must match for equal objects when using hashing collections (HashMap/HashSet)

If you want, I can also give you:
✅ **All methods of Object class related to equality**
✅ A **custom Employee class** with proper equals/hashCode/toString ready for interview
✅ A **diagram: HashMap bucket → equals check flow**